

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3

Выполнила:

Мищенко Арина

Группа J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Теоретическая часть

1. Vue.js

Vue.js – это прогрессивный JavaScript-фреймворк для создания пользовательских интерфейсов. Слово «прогрессивный» означает, что Vue можно внедрять постепенно: начать с подключения одного скрипта на страницу, а затем масштабироваться до полноценного SPA (Single Page Application — одностраничное приложение). SPA – веб-приложение, где переходы между страницами происходят без перезагрузки браузера: JavaScript-роутер перехватывает навигацию и динамически меняет отображаемый компонент.

Основные преимущества Vue перед «ванильным» JavaScript и подходом из ЛР1/ЛР2: реактивность (DOM автоматически обновляется при изменении данных), компонентный подход (интерфейс делится на переиспользуемые независимые части), декларативный стиль (описываем ЧТО должно отображаться, а не КАК обновлять DOM).

2. Однофайловые компоненты (SFC)

Vue-компонент оформляется как файл с расширением `.vue` — Single File Component. Файл состоит из трёх секций:

- `<template>` – HTML-разметка компонента с Vue-директивами.
- `<script>` или `<script setup>` – JavaScript-логика: данные, методы, хуки жизненного цикла.
- `<style scoped>` – CSS, область видимости которого ограничена данным компонентом.

Деление на компоненты в проекте: Views (представления) – страницы, на которые ведёт роутер; Components – переиспользуемые части страниц (навбар, карточка курса, фильтры); Layouts – обёртки со слотом для общей структуры страниц.

3. Options API и Composition API

Vue 3 предоставляет два API для написания логики компонентов.

Options API (использован в компонентах TheNavbar, CourseCard): логика организована по типу опций — `data()` для данных, `computed` для вычисляемых свойств, `methods` для методов, `mounted()` для хуков. Удобен для простых компонентов с ограниченной логикой.

Composition API с `<script setup>` (использован в Views): логика организована по функциональным блокам, а не по типу опций. Используются примитивы: `ref(value)` – реактивная ссылка, значение читается через `.value`; `computed(() => ...)` – вычисляемое свойство; `onMounted(() => ...)` – хук монтирования. Ключевое преимущество: Composition API позволяет естественно использовать composables.

4. Composables

`Composable` – это функция с префиксом `use`, которая использует `Composition API` для инкапсуляции и переиспользования логики с состоянием. В отличие от обычных утилит, `composable` содержит реактивное состояние через `ref` и `computed`.

`Composables` решают проблему дублирования кода: если несколько компонентов имеют одинаковую логику (например, управление флагом загрузки или фильтрация списка), эта логика выносится в `composable` и импортируется там, где нужна.

```
// Пример: использование двух composables в одном компоненте
const { loading, withLoading } = useLoading()
const { filteredCourses, searchQuery } = useFilters(courses)
```

5. Vue Router

`Vue Router` – официальная библиотека маршрутизации. Конфигурация определяет соответствие URL-путей и компонентов-представлений. В корневом `App.vue` размещается `<router-view />` – точка монтирования активного представления. Ссылки создаются через `<router-link to="/path">` вместо тега `<a>`.

Навигационный `guard` (`router.beforeEach`) – функция, вызываемая перед каждым переходом. Это `Vue`-аналог проверки `sessionStorage` в начале каждого HTML-файла из ЛР2: если пользователь не авторизован и пытается открыть защищённую страницу, `guard` перенаправляет его на страницу входа.

Параметры маршрута читаются через `useRoute().params.id` (`Composition API`). Динамический маршрут задаётся как `path: '/courses/:id'`.

6. Pinia — управление состоянием

`Pinia` – официальный менеджер состояния `Vue 3`. Хранилище создаётся функцией `defineStore()` и содержит: `state` (данные, аналог `data()`), `getters` (вычисляемые свойства), `actions` (методы). Параметр `persist: true` через плагин `pinia-persist` автоматически сохраняет состояние в `localStorage` – это заменяет ручную работу с `sessionStorage` из ЛР2.

Для реактивного доступа к данным хранилища в `Composition API` используется функция `storeToRefs(store)` – она превращает свойства хранилища в реактивные ссылки, которые можно деструктурировать без потери реактивности.

Задача

В рамках лабораторной работы №3 необходимо мигрировать ранее разработанное веб-приложение платформы для онлайн-курсов (ЛР1 + ЛР2) на фреймворк `Vue.js` в формате одностраничного приложения (SPA). Требования:

- Подключён `Vue Router` с навигационным `guard` для защиты страниц авторизованного доступа.
- Реализована работа с внешним API посредством `axios`.
- Разумное деление на компоненты: `Views`, `Components`, `Layouts`.
- Использование `composables` для выделения повторяющегося функционала.

Ход работы

1. Структура проекта

Проект создан командой `npm init vue@latest` с подключением Router и Pinia. Дополнительно установлены: `axios` (HTTP-запросы), `bootstrap` и `bootstrap-icons` (стили), `pinia-persists` (сохранение состояния). Добавлены папки `src/composables/` и `src/layouts/` сверх стандартной структуры.

Миграция затронула все страницы исходного приложения: `index.html` - `HomePage.vue`, `login.html` - `LoginPage.vue`, `register.html` - `RegisterPage.vue`, `search.html` - `CoursesPage.vue`, `course.html` - `CoursePage.vue`, `dashboard-user.html` - `DashboardPage.vue`, `dashboard-teacher.html` - `TeacherPage.vue`.

2. API-слой (`src/api/`)

Реализован по принципу IoC-контейнера из мануала. Для каждой коллекции `json-server` создан отдельный класс, принимающий `axios`-инстанс через конструктор: `CoursesApi`, `AuthApi`, `EnrollmentsApi`, `TasksApi`, `CompletedTasksApi`, `DiscussionsApi`. Файл `src/api/index.js` создаёт единственные экземпляры всех классов. Такой подход позволяет в любой момент заменить `axios` на другую библиотеку, изменив только `instance.js`.

3. Хранилища Pinia (`src/stores/`)

Создано три хранилища. `useAuthStore` с параметром `persist: true` заменяет работу с `sessionStorage` из ЛР2: данные пользователя (`id`, имя, роль) теперь хранятся в `localStorage` и автоматически восстанавливаются при обновлении страницы. `useCoursesStore` загружает только опубликованные курсы (аналогично фильтру в `search.html`).

4. Composables (`src/composables/`)

Созданы три `composable`, устраняющих дублирование кода между страницами:

`useLoading.js` – управление состоянием загрузки. Содержит реактивные переменные `loading` и `error` и функцию `withLoading(fn)`, которая оборачивает любую `async`-функцию, автоматически устанавливает флаг загрузки и перехватывает ошибки. Используется на всех семи страницах, заменяя дублирующийся `try/catch`-код из ЛР2.

```
// Было в каждой HTML-странице:
submitBtn.disabled = true; submitBtn.textContent = 'Вход...';
try { ... } finally { submitBtn.disabled = false; }

// Стало (composable):
const { loading, withLoading } = useLoading()
withLoading(() => login(email, password, role))
```

`useFilters.js` – логика фильтрации курсов из `search.html`, вынесенная в отдельный файл. Принимает реактивный массив курсов, возвращает `ref`-переменные фильтров и

вычисляемое свойство `filteredCourses`. Фильтрация пересчитывается автоматически при изменении любого фильтра – благодаря `computed()`, что заменяет ручной вызов `applyFilters()` из ЛР2.

useAuth.js – инкапсулирует работу с `authStore` и навигацию после входа/выхода. Использует `storeToRefs()` для реактивного доступа. Применяется в `TheNavbar`, `LoginPage`, `RegisterPage` и `HomePage`, обеспечивая единое поведение без дублирования.

5. Компоненты (src/components/, src/layouts/)

Реализованы четыре компонента:

TheNavbar.vue – адаптивная навигационная панель. Используется через `BaseLayout` на всех страницах, кроме авторизационных. Использует `setup()` внутри `Options API` для подключения `useAuth`, показывает разный контент для гостя, студента и преподавателя через `v-if/v-else`.

CourseCard.vue – карточка курса. Принимает данные через props (`id`, `title`, `teacher`, `level`, `price`, `lessons`, `hours`, `rating`, `reviews`, `color`, `icon`). Используется в `CoursesPage` через `v-bind="course"` — передача всех полей объекта как props одной директивой.

BaseLayout.vue – обёртка с навбаром и контейнером. Принцип слота: `<slot />` – точка монтирования для содержимого страницы. Устраняет дублирование HTML навбара в каждом файле.

AuthLayout.vue – лейаут для страниц входа и регистрации без основного навбара, с центрированной карточкой.

6. Маршрутизация (src/router/index.js)

Настроены 7 маршрутов. Маршруты `/dashboard` и `/teacher` помечены метаданными `requiresAuth: true` и `role`. Навигационный `guard router.beforeEach` проверяет авторизацию и роль пользователя перед каждым переходом. Это Vue-эквивалент проверки `sessionStorage.getItem('role')` в начале каждого HTML-файла из ЛР2.

7. Страница курса (CoursePage.vue)

Наиболее сложная страница – мигрирует логику из `course.html`. Состояния заданий хранятся в реактивном объекте `taskStateMap` (как в оригинале). Четыре возможных состояния задания (не сдано / на проверке / принято / на доработку) реализованы через вычисляемые функции `taskIcon()`, `taskBtnText()`, `taskBtnClass()`. Обсуждения загружаются через `discussionsApi`, запись на курс через `enrollmentsApi`.

8. Кабинет преподавателя (TeacherPage.vue)

Мигрирует логику из `dashboard-teacher.html`. При проверке задания студента (кнопки «Принять» / «На доработку») вызывается `reviewTask(ct, result)`, которая: PATCH `completedTask`, пересчитывает количество принятых заданий, PATCH `enrollment` с новым прогрессом – аналогично оригиналу. Создание и редактирование курсов реализовано через форму в модальном окне.

Вывод

В ходе лабораторной работы приложение платформы для онлайн-курсов EduPlatform полностью мигрировано на Vue.js в формате одностраничного приложения. Все семь HTML-страниц переработаны в Vue-компоненты. Реализованы все четыре требования: Vue Router с навигационным guard, API-слой на axios с IoC-архитектурой, компонентное деление на Views/Components/Layouts, три composable (useLoading, useFilters, useAuth) для устранения дублирования кода. Применены оба API Vue 3: Options API в компонентах и Composition API с `<script setup>` в представлениях.